

Mathematics and Statistics

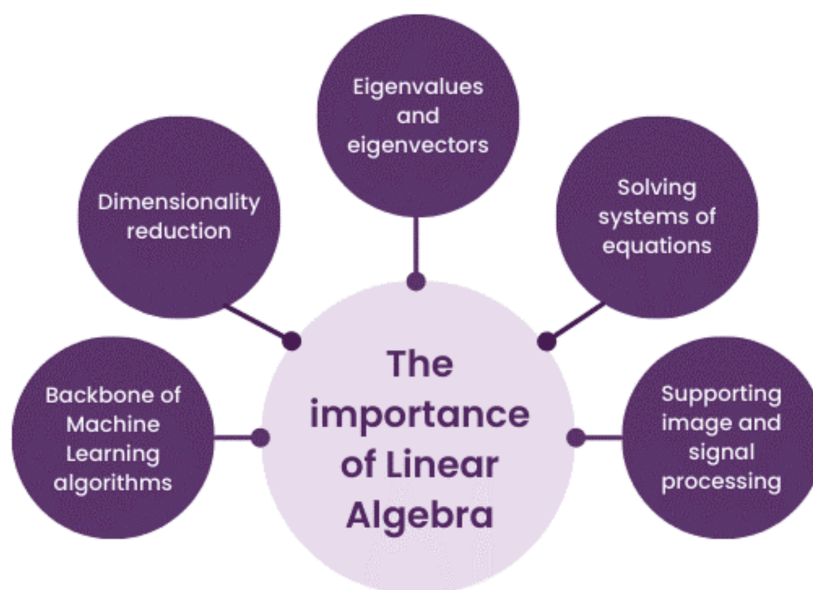
Linear algebra, probability, and statistics are the core mathematical foundations for data science, enabling the transformation of raw data into actionable insights through Exploratory Data Analysis (EDA) and machine learning. They allow analysts to represent data structurally, manage uncertainty, and summarize patterns.

Linear Algebra

Linear algebra is the language of data manipulation, providing the framework for organizing, transforming, and compressing large datasets. Linear Algebra plays a foundational role in the realm of Data Science. It provides the mathematical underpinning for various data manipulation, machine learning, and deep learning techniques.

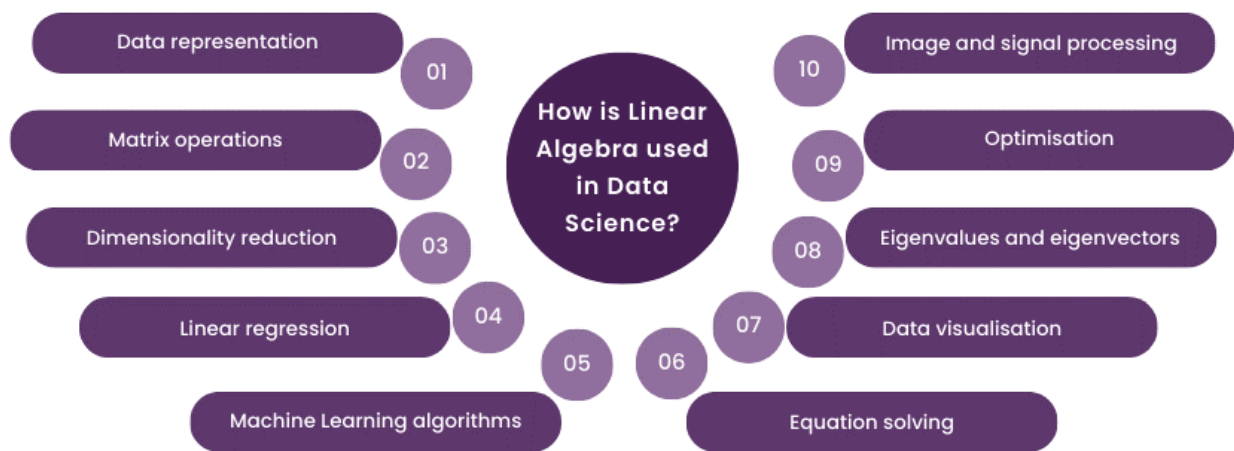
To illustrate the significance of Linear Algebra in Data Science, let's take the example of image recognition. While humans can effortlessly perceive an image, computers face a more challenging task. The process of image recognition heavily relies on matrix-based tools.

In the digital realm, images are composed of pixels, the smallest indivisible elements. Each pixel is characterized by an intensity value. Consequently, an image is essentially a matrix, with each pixel's intensity values constituting its elements. Linear Algebra comes into play when expanding, compressing, cropping, or performing any image manipulation. It forms the foundation of deep learning and machine learning. The importance of Linear Algebra in Data Science is multiple and are listed as shown image below.



- **Backbone of Machine Learning algorithms:** Linear Algebra enables fundamental operations like matrix multiplication, essential for model training and predictions.
- **Dimensionality Reduction:** Linear Algebra techniques facilitate dimensionality reduction, enhancing data processing and interpretation.
- **Eigenvalues and Eigenvectors:** These concepts help in understanding variability in data records, impacting clustering and pattern recognition.
- **Solving Systems of Equations:** Crucial for optimization tasks and parameter estimation.
- **Supporting Image and Signal Processing:** Linear Algebra is vital for image and signal processing strategies and is critical in data analysis.

Linear Algebra empowers data scientists to effectively represent, manipulate, and extract insights from data, ultimately driving the development of accurate models and informed decision-making. Linear Algebra serves as the foundational stone to perform big data analysis in the Data Science domain. Here are the few essential applications of Linear Algebra in Data Science as listed image below.



1. Data Representation: Matrices are commonly used to represent data, with rows denoting observations and columns representing features, allowing for efficient manipulation and analysis.

2. Matrix Operations: Fundamental operations like addition, multiplication, and transposition play crucial roles in various calculations, including similarity measures, data transformations, and equation solving.

3. Dimensionality Reduction: Techniques like Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) leverage Linear Algebra to simplify complex data while preserving crucial information.

4. Linear Regression: Linear Algebra serves as the foundation for linear regression, a prevalent method for modelling relationships between variables and making predictions.

5. Machine Learning Algorithms: Linear Algebra operations are essential in algorithms like support vector machines, linear discriminant analysis, and logistic regression for model building and data classification.

6. Image and signal processing: Image tasks like filtering, compression, and edge detection heavily rely on Linear Algebra, with operations such as Fourier transforms and convolutions.

7. Optimization: Linear Algebra is pivotal in optimization algorithms, including gradient descent, which is fundamental for calculating gradients in machine learning.

8. Eigenvalues and Eigenvectors: These concepts are crucial for recognizing significant data patterns and variability directions, benefiting clustering, feature extraction, and data characterization.

9. Data Visualisation: Linear Algebra-based dimensionality reduction techniques, like PCA, aid in visualizing high-dimensional data in lower-dimensional spaces.

10. Equation Solving: Linear Algebra techniques are regularly used to solve sets of linear equations, commonly encountered in optimization problems and parameter estimation

Getting a good handle on linear algebra for data science is like picking up a really powerful tool it helps you make sense of data manipulation, machine learning, and deep learning. Whether you're just learning the basics or diving into more advanced applications, it's all about using linear algebra to its full potential.

Probability

Probability serves as the foundational mathematics for Exploratory Data Analysis (EDA) and data science, allowing practitioners to quantify uncertainty, model data generation processes, and distinguish signal from noise. While EDA focuses on visualizing and summarizing data to discover patterns, probability provides the theoretical framework to understand the likelihood of those patterns occurring.

To understand probability, we must first define its basic components:

- **Sample Space (Ω):** The set of all possible outcomes from a random experiment
- **Event:** A subset of the sample space

- Outcome: A single element of the sample space

Example : Rolling Dice



When rolling a standard six-sided die, the sample space is $\Omega = \{1, 2, 3, 4, 5, 6\}$. Various events can be defined from this sample space:

- Event A: “Rolling an even number” = $\{2, 4, 6\}$
- Event B: “Rolling a number greater than 4” = $\{5, 6\}$
- Event C: “Rolling a prime number” = $\{2, 3, 5\}$

Example: Market Analysis

A market analyst studying smartphone purchases finds:

- $P(\text{purchases iPhone}) = 0.45$
- $P(\text{purchases Android}) = 0.50$
- $P(\text{purchases both iPhone and Android}) = 0.05$

The probability that a consumer purchases either an iPhone or an Android smartphone is:

$$P(\text{iPhone} \cup \text{Android}) = P(\text{iPhone}) + P(\text{Android}) - P(\text{iPhone} \cap \text{Android}) \\ = 0.45 + 0.50 - 0.05 = 0.90$$

This means 90% of consumers purchase at least one of these smartphone types, while 10% purchase neither.

Probability Axioms and Properties

Modern probability theory is built on three fundamental axioms introduced by Kolmogorov:

- **Non-negativity:** For any event A, $P(A) \geq 0$
- **Normalization:** $P(\Omega) = 1$ (the probability of the entire sample space is 1)
- **Additivity:** For mutually exclusive events A and B, $P(A \cup B) = P(A) + P(B)$

From these axioms, we can derive several important properties:

- **Complement Rule:** $P(A^c) = 1 - P(A)$

- **Monotonicity:** If $A \subseteq B$, then $P(A) \leq P(B)$
- **Probability Bounds:** For any event A , $0 \leq P(A) \leq 1$
- **General Addition Rule:** $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Counting Techniques in Probability

Many probability problems require counting the number of possible outcomes or events.

Several techniques are essential:

Permutations

Arrangements where order matters. The number of permutations of n distinct objects is:

$$P(n) = n!$$

For selecting and arranging r objects from n distinct objects:

$$P(n,r) = n!/(n-r)!$$

Example : Product Configuration

A manufacturer offers customers the ability to select 3 features from 7 available options, where the order of selection affects the final product. The number of possible configurations is:

$$P(7,3) = 7!/(7-3)! = 7!/4! = 7 \times 6 \times 5 = 210$$

Combinations

Selections where order doesn't matter. The number of ways to select r objects from n distinct objects is:

$$C(n,r) = n!/(r!(n-r)!)$$

Example : Team Formation

From a pool of 20 qualified data scientists, a company needs to form a team of 5 for a project.

The number of possible team combinations is:

$$C(20,5) = 20!/(5!(20-5)!) = 20!/(5!15!) = 15,504$$

Understanding these counting principles is crucial for correctly calculating probabilities in complex scenarios with multiple possible outcomes.

Types of Probability

Three main interpretations of probability are commonly used, each with its strengths and appropriate applications:

Classical Probability

Classical probability applies when all outcomes in the sample space are equally likely. It's calculated as:

$$P(A) = \text{Number of favorable outcomes} / \text{Total number of possible outcomes}$$

This interpretation is particularly useful for well-defined chance experiments like coin flips, dice rolls, and card draws.

Example : Poker Hands

In a standard 52-card deck, the probability of being dealt a flush (5 cards of the same suit) in a 5-card poker hand is calculated as:

$P(\text{flush}) = \text{Number of flush hands} / \text{Total number of possible hands}$

There are $C(13,5)$ ways to select 5 cards from a single suit, and there are 4 suits, giving us $4 \times C(13,5) = 4 \times 1,287 = 5,148$ flush hands.

The total number of possible 5-card hands is $C(52,5) = 2,598,960$.

Therefore: $P(\text{flush}) = 5,148 / 2,598,960 \approx 0.00198$ or about 0.2%

Frequentist Probability

Frequentist probability defines probability as the long-run relative frequency of an event occurring in repeated trials under identical conditions. Mathematically:

$$P(A) = \lim_{n \rightarrow \infty} n_A / n$$

Where n_A is the number of times event A occurs in n trials.

This interpretation is particularly useful in experimental sciences and data analysis, where probabilities are estimated from observed frequencies.

Example: Website Conversion Rates

An e-commerce website tracks that out of 25,000 visitors, 875 completed a purchase. The frequentist probability of conversion is:

$$P(\text{conversion}) = 875 / 25,000 = 0.035 \text{ or } 3.5\%$$

If we segment the data, we might find that:

- Desktop users: 500 conversions out of 12,000 visitors $\rightarrow P(\text{conversion}|\text{desktop}) = 500 / 12,000 = 0.0417$ or 4.17%
- Mobile users: 375 conversions out of 13,000 visitors $\rightarrow P(\text{conversion}|\text{mobile}) = 375 / 13,000 = 0.0288$ or 2.88%

This segmentation reveals that desktop users have a higher conversion rate, which could inform website optimization strategies.

Subjective (Bayesian) Probability

Subjective probability represents a degree of belief about an event, often incorporating prior knowledge, expert judgment, or subjective assessment. Unlike classical and frequentist

interpretations, subjective probability acknowledges that different individuals might assign different probabilities to the same event based on their knowledge and beliefs.

Example: New Product Launch Success

A product manager might estimate a 70% probability of a successful product launch based on:

- Market research data
- Industry experience
- Performance of similar previous products
- Competitive landscape assessment

Another manager with different experience or information might estimate a different probability. The Bayesian approach provides a framework for systematically updating these subjective probabilities as new evidence becomes available.

Subjective probability forms the foundation of Bayesian statistics, which has gained significant traction in data science for its ability to incorporate prior knowledge and update beliefs based on evidence.

Descriptive/Inferential Statistics

Descriptive statistics are the foundational techniques for summarizing, organizing, and visualizing data in exploratory data analysis (EDA). They involve calculating key metrics like central tendency (mean, median, mode), spread (variance, standard deviation, IQR), and frequency distributions to identify patterns, inconsistencies, and outliers before applying advanced analytics.

Key Components of Descriptive Statistics for EDA

- **Measures of Central Tendency:** These indicate the center of the data distribution, including the mean (average), median (middle value), and mode (most frequent).
- **Measures of Dispersion (Spread):** These determine how spread out the data points are. Key metrics include variance, standard deviation (average distance from the mean), and range.
- **Measures of Shape/Frequency:** These reveal the distribution structure, such as skewness (asymmetry) and kurtosis (tailedness).
- **Position Measures:** These describe the relative standing of data points, such as percentiles and quartiles, commonly visualized using box plots.

Measures of Central Tendency

A measure of central tendency is a single value that attempts to describe a set of data by identifying the central position within that set of data. As such, measures of central tendency are sometimes called measures of central location. They are also classed as summary statistics. The mean (often called the average) is most likely the measure of central tendency that you are most familiar with, but there are others, such as the median and the mode.

The mean, median and mode are all valid measures of central tendency, but under different conditions, some measures of central tendency become more appropriate to use than others. In the following sections, we will look at the mean, mode and median, and learn how to calculate them and under what conditions they are most appropriate to be used.

Mean (Arithmetic)

The mean (or average) is the most popular and well-known measure of central tendency. It can be used with both discrete and continuous data, although its use is most often with continuous data (see our Types of Variable guide for data types). The mean is equal to the sum of all the values in the data set divided by the number of values in the data set.

Median

The median is the middle score for a set of data that has been arranged in order of magnitude. The median is less affected by outliers and skewed data. In order to calculate the median, suppose we have the data below:

65	55	89	56	35	14	56	55	87	45	92
----	----	----	----	----	----	----	----	----	----	----

We first need to rearrange that data into order of magnitude (smallest first):

14	35	45	55	55	56	56	65	87	89	92
----	----	----	----	----	-----------	----	----	----	----	----

Our median mark is the middle mark - in this case, 56 (highlighted in bold). It is the middle mark because there are 5 scores before it and 5 scores after it. This works fine when you have an odd number of scores, but what happens when you have an even number of scores? What if you had only 10 scores? Well, you simply have to take the middle two scores and average the result. So, if we look at the example below:

65	55	89	56	35	14	56	55	87	45
----	----	----	----	----	----	----	----	----	----

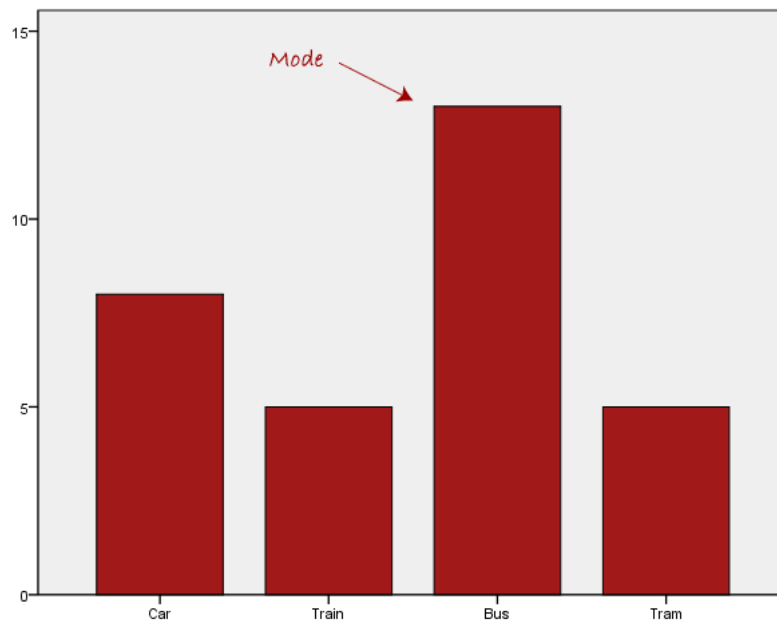
We again rearrange that data into order of magnitude (smallest first):

14	35	45	55	55	56	56	65	87	89
----	----	----	----	-----------	-----------	----	----	----	----

Only now we have to take the 5th and 6th score in our data set and average them to get a median of 55.5.

Mode

The mode is the most frequent score in our data set. On a histogram it represents the highest bar in a bar chart or histogram. You can, therefore, sometimes consider the mode as being the most popular option. Normally, the mode is used for categorical data where we wish to know which is the most common category.



We can see above that the most common form of transport, in this particular data set, is the bus. However, one of the problems with the mode is that it is not unique, so it leaves us with problems when we have two or more values that share the highest frequency.

Measures of Dispersion

Measures of dispersion, also called measures of variability, "describe the extent to which the values of a variable are different." The most common measures of dispersion are range, variance, standard deviation, and the coefficient of variation.

Range

Range is a very simple statistic, as it is merely the difference between the largest and smallest values. It's not very useful from a statistical standpoint; because it relies only on the outermost values of a dataset, two datasets with the same range could still have drastic differences in their overall distribution.

Variance

Variance can refer to two different things: population variance (sometimes called parametric variance) and sample variance. Population variance, typically represented by a lower-case

sigma squared, can only be truly calculated if you have observations for every member of your population, which is almost never the case. It's used more often by statisticians and using it should never be your first instinct.

Standard Deviation

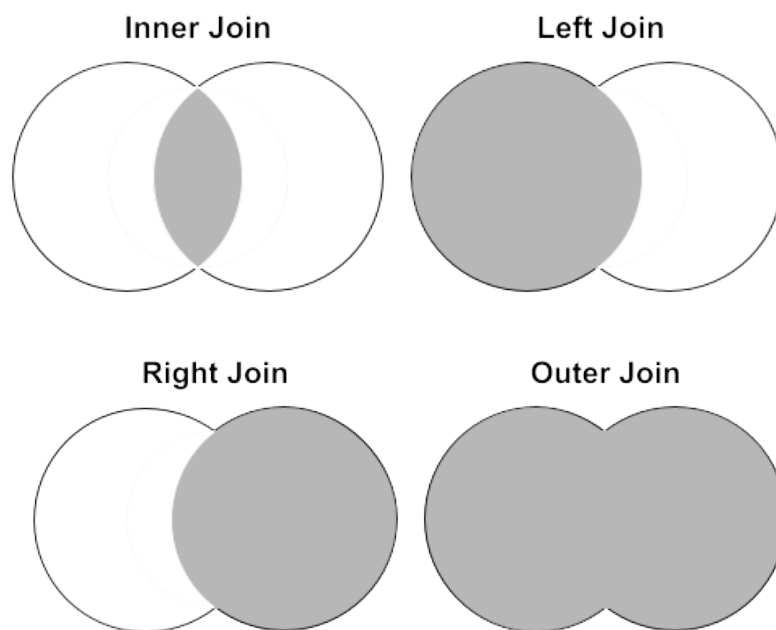
Like variance, standard deviation can theoretically be calculated for both a population and a sample. The population standard deviation is rarely used; sample standard deviation (s) should be your first choice. Easy to calculate, it is simply the square root of the variance. It tends to be more understandable than variance because it expresses the dispersion of a dataset in that dataset's original units.

SQL & Databases

Merging and Joining Datasets

Merging datasets is essential when working with data from multiple sources. This technique allows you to combine information based on common fields, such as IDs or dates. There are several types of merges:

- Inner Join: Returns rows with matching values in both datasets.
- Left Join: Returns all rows from the left dataset and matching rows from the right dataset.
- Right Join: Returns all rows from the right dataset and matching rows from the left dataset.
- Outer Join: Returns all rows when there is a match in either dataset.



For example, let's consider these two datasets for an e-commerce store. One dataset contains order information, and the other contains customer data.

```
# Load the pandas package and create the dataset
import pandas as pd
data = {
    'Order_ID': ['Order - 101', 'Order - 102', 'Order - 103', 'Order - 104',
                 'Order - 105'],
    'Customer_ID': ['Customer - 1', 'Customer - 2', 'Customer - 3', 'Customer - 1', 'Customer - 4'],
```

```

    'Order_Date ': ['2025-10-26', '2025-10-27', '2025-10-28', '2025-10-29'
, '2025-10-30'],
    'Amount': [100, 50, 75, 120, 80]
}
df_order = pd.DataFrame(data)
df_order

```

Dataset 1: Order sample data

Order_id	Customer_ID	Order_Date	Amount
Order – 101	Customer – 1	2023-10-26	100
Order – 102	Customer – 2	2023-10-27	50
Order – 103	Customer – 3	2023-10-28	75
Order – 104	Customer – 1	2023-10-29	120
Order – 105	Customer – 4	2023-10-30	80

Dataset 2: Customer sample data

```

# Load the pandas package and create the dataset
import pandas as pd
data = {
    'Customer_ID': ['Customer - 1', 'Customer - 2', 'Customer - 3', 'Customer - 5'],
    'Customer_Name': ['Hailesllassie Zeferu Abel', 'Worku Abenezer Girma', 'Abenezer Reta Tamirat', 'Abraham Melisew Fireda'],
    'City': ['Addis Ababa', 'Jimma', 'Mekelle', 'Gondar']
}
df_customer = pd.DataFrame(data)
df_customer

```

Customer_ID	Customer Name	City
Customer - 1	Hailesllassie Zeferu Abel	Addis Ababa
Customer - 2	Worku Abenezer Girma	Jimma
Customer - 3	Abenezer Reta Tamirat	Mekelle
Customer - 5	Abraham Melisew Fireda	Gondar

The two datasets (Order and Customer) data can be merged based on the shared column (Customer_ID) in four different ways:

Inner Join

An inner join would only return rows where the Customer_ID exists in both the first (i.e., Order) and second (i.e., Customer) datasets. This would exclude:

- Orders made by unregistered customers (e.g., Order 105).
- Customers who haven't placed any orders (e.g., David).

```
import pandas as pd
inner_join = pd.merge(df_order, df_customer, on='Customer_ID', how='inner')
print(inner_join)
```

	Order ID	Customer ID	Order Date	Amount	Customer Name	City
0	Order - 101	Customer - 1	2023-10-26	100	Hailelassie Zeferu Abel	Addis Ababa
1	Order - 104	Customer - 1	2023-10-27	120	Hailelassie Zeferu Abel	Addis Ababa
2	Order - 102	Customer - 2	2023-10-28	50	Worku Abenezer Girma	Jimma
3	Order - 103	Customer - 3	2023-10-29	75	Abenezer Reta Tamirat	Mekelle

Left Join

A left join will keep all records from the order dataset, even if there is no matching customer information. This is useful if you are focused on analyzing orders and want to see if there are any orders from customers who are not registered in your system (for example, guest checkouts or unregistered customers).

```
import pandas as pd
left_join = pd.merge(df_order, df_customer, on='Customer_ID',
how='left')
left_join
```

	Order ID	Customer ID	Order Date	Amount	Customer Name	City
0	Order - 101	Customer - 1	2023-10-26	100	Hailelassie Zeferu Abel	Addis Ababa
1	Order - 102	Customer - 2	2023-10-27	50	Worku Abenezer Girma	Jimma
2	Order - 103	Customer - 3	2023-10-28	75	Abenezer Reta Tamirat	Mekelle
3	Order - 104	Customer - 1	2023-10-29	120	Hailelassie Zeferu Abel	Addis Ababa
4	Order - 105	Customer - 4	2023-10-30	80	NaN	NaN

Right Join

A right join will keep all records from the Customer Information dataset, including customers who haven't placed any orders yet. This can be valuable if you want to track customers who are registered but haven't made any purchases, allowing for customer re-engagement efforts.

```
import pandas as pd
right_join = pd.merge(df_order, df_customer, on='Customer_ID',
```

```
how='right')
print(right_join)
```

	Order_ID	Customer_ID	Order_Date	Amount	Customer_Name	City
0	Order - 101	Customer - 1	2023-10-26	100	Hailelassie Zeferu Abel	Addis Ababa
1	Order - 104	Customer - 1	2023-10-29	120	Hailelassie Zeferu Abel	Addis Ababa
2	Order - 102	Customer - 2	2023-10-27	50	Worku Abenezer Girma	Jimma
3	Order - 103	Customer - 3	2023-10-28	75	Abenezer Reta Tamirat	Mekelle
4	NaN	Customer - 5	NaN	NaN	Abraham Melisew Fireda	Gondar

Outer Join

An outer join will include all records from both datasets, even if there are no matching records in the other dataset. This is valuable for a comprehensive overview, showing both unregistered customers and customers who haven't placed orders.

```
import pandas as pd
outer_join = pd.merge(df_order, df_customer, on='Customer_ID',
how='outer')
print(outer_join)
```

	Order_ID	Customer_ID	Order_Date	Amount	Customer_Name	City
0	Order - 101	Customer - 1	2025-10-26	100	Hailelassie Zeferu Abel	Addis Ababa
1	Order - 104	Customer - 1	2025-10-29	120	Hailelassie Zeferu Abel	Addis Ababa
2	Order - 102	Customer - 2	2025-10-27	50	Worku Abenezer Girma	Jimma
3	Order - 103	Customer - 3	2025-10-28	75	Abenezer Reta Tamirat	Mekelle
4	Order - 105	Customer - 4	2025-10-30	80	NaN	NaN
5	NaN	Customer - 5	NaN	NaN	Abraham Melisew Fireda	Gondar

As you may have noticed, apart from an Inner Join, all other merge types can produce rows with missing values (represented by "NaN" in the tables above). Therefore, it is important to choose the most appropriate join based on your specific needs.

Filtering and Sub-setting Data

Filtering and sub-setting involve selecting a specific subset of your data based on conditions or criteria. This technique is crucial for focusing your analysis on the most relevant data. For example, let's consider this dataset that contains product sales records. Your goal is to filter or subset the data to analyze sales based on specific criteria like product category, date, or total sales amount.

Sales Data

```
# Load the pandas package and create the dataset
```

```
import pandas as pd

data = {
    'Order_ID': ['1001', '1002', '1003', '1004', '1005'],
    'Product_ID': ['P101', 'P102', 'P103', 'P101', 'P104'],
    'Product_Name': ['Laptop', 'Smartphone', 'Headphones', 'Laptop',
'Coffee Maker'],
    'Category': ['Electronics', 'Electronics', 'Accessories',
'Electronics', 'Kitchen'],
    'Quantity': [2, 1, 5, 3, 2],
    'Amount': [1600, 600, 500, 2400, 100],
    'Date': ['2024-01-10', '2024-01-11', '2024-01-12', '2024-01-12',
'2024-01-13'],
    'Region': ['North', 'South', 'East', 'West', 'North'],
}

df_sales = pd.DataFrame(data)
df_sales
```

Order ID	Product ID	Product Name	Category	Quantity	Amount	Date	Region
1001	P101	Laptop	Electronics	2	1,600	2024-01-10	North
1002	P102	Smartphone	Electronics	1	600	2024-01-11	South
1003	P103	Headphones	Accessories	5	500	2024-01-12	East
1004	P101	Laptop	Electronics	3	2,400	2024-01-12	West
1005	P104	Coffee Maker	Kitchen	2	100	2024-01-13	North

You may want to filter the dataset to show only sales related to **Electronics** products, so you filter your dataset by Category.

```
import pandas as pd
electronics_sales = df_sales[df_sales['Category'] == 'Electronics']
electronics_sales
```

	Order ID	Product ID	Product Name	Category	Quantity	Amount	Date	Region
0	1001	P101	Laptop	Electronics	2	1600	2024-01-10	North
1	1002	P102	Smartphone	Electronics	1	600	2024-01-11	South
3	1004	P101	Laptop	Electronics	3	2400	2024-01-12	West

Let's say you want to filter sales that are greater than 1,000.

Filtered by Total Amount (>1,000)

```
import pandas as pd
big_sales = df_sales[df_sales ['Amount'] > 1000]
big_sales
```

Filtered by Total Amount (>1000) AND Date (Before Jan 12, 2024)

	Order ID	Product ID	Product Name	Category	Quantity	Amount	Date	Region
0	1001	P101	Laptop	Electronics	2	1600	2024-01-10	North
3	1004	P101	Laptop	Electronics	3	2400	2024-01-12	West

```
import pandas as pd
filtered_df = df_sales[(df_sales ['Amount'] > 1000) &
(pd.to_datetime(df_sales['Date']) < '2024-01-12')]
filtered_df
```

	Order ID	Product ID	Product Name	Category	Quantity	Amount	Date	Region
0	1001	P101	Laptop	Electronics	2	1600	2024-01-10	North

Subset by Specific Columns (Order ID, Product Name, and Total Amount)

```
import pandas as pd
subset_df = df_sales[['Order_ID', 'Product_Name', 'Amount']]
print(subset_df)
```

	Order ID	Product Name	Amount
0	1001	Laptop	1904-05-18
1	1002	Smartphone	1901-08-22
2	1003	Headphones	1901-05-14
3	1004	Laptop	1906-07-27
4	1005	Coffee Maker	1900-04-09

Aggregating and Transforming Data

Aggregation involves summarizing data, such as calculating averages, sums, or counts. Transforming data includes creating new calculated fields, standardizing data, or applying mathematical functions. Let's consider this new dataset where students have scores in multiple subjects.

Student Test Results

```
# Load the pandas package and create the dataset
import pandas as pd
data = {
    'Student_ID': ['S001', 'S002', 'S003', 'S004', 'S005'],
    'Mathematics' : [85, 75, 95, 65, 90],
```



```

    'Science' : [78, 88, 82, 85, 79],
    'English' : [90, 85, 80, 70, 95],
    'Information Technology' : [89, 65, 88, 92, 98]
}
df_result = pd.DataFrame(data)
df_result

```

Student ID	Math	Science	English	IT
S001	85	78	90	89
S002	75	88	85	65
S003	95	82	80	88
S004	65	85	70	92
S005	90	79	95	98

With that dataset, we will perform the following actions:

1. Creating new calculated fields: Calculating the average and weighted average of test scores.
2. Standardizing data: Standardizing the test scores.
3. Applying functions: Assigning final grades based on weighted average

Average

```

import numpy as np

df_result['Average'] = np.mean(df_result[['Mathematics', 'Science',
'English', 'Information Technology']], axis=1)

df_result

```

	Student ID	Mathematics	Science	English	Information Technology	Average
0	S001	85	78	90	89	85.5
1	S002	75	88	85	65	78.25
2	S003	95	82	80	88	86.25
3	S004	65	85	70	92	78
4	S005	90	79	95	98	90.5

Weighted Average

Suppose the Math subject is weighted slightly more (0.4) than Science and English (0.3 each).

We can calculate the weighted average and display the results in a new column:

```
import numpy as np

df_result['Weighted Average'] = (df_result['Mathematics'] * 0.4) +
(df_result['Science'] * 0.2) + (df_result['English'] * 0.2) +
(df_result['Information Technology'] * 0.2)
df_result
```

	Student ID	Mathematics	Science	English	Information Technology	Average	Weighted Average
0	S001	85	78	90	89	85.5	85.4
1	S002	75	88	85	65	78.25	77.6
2	S003	95	82	80	88	86.25	88
3	S004	65	85	70	92	78	75.4
4	S005	90	79	95	98	90.5	90.4

Standardized Scores

We can also calculate the standardized value for each test score. Standardization is the process of transforming data to have a mean of 0 and a standard deviation of 1. This technique is especially useful when comparing variables that have different units or scales. Standardizing the data ensures that each variable contributes equally to the analysis, making comparisons more meaningful.

```
import numpy as np

# Standardization function
def standardize(column):
    mean = np.mean(column)
    std_dev = np.std(column)
    return (column - mean) / std_dev

# Standardize the columns
df_result['Math_Std'] = standardize(df_result['Mathematics'])
df_result['Sci_Std'] = standardize(df_result['Science'])
df_result['Eng_Std'] = standardize(df_result['English'])
df_result['IT_Std'] = standardize(df_result['Information Technology'])
df_result
```

	Student ID	Mathematics	Science	English	Information Technology	Average	Weighted Average	Math_Std	Sci_Std	Eng_Std	IT_Std
0	S001	85	78	90	89	85.5	85.4	0.278543	-1.182727	0.697486	0.23104
1	S002	75	88	85	65	78.25	77.6	-0.649934	1.505289	0.116248	-1.90164
2	S003	95	82	80	88	86.25	88	1.20702	-0.107521	0.464991	0.142179
3	S004	65	85	70	92	78	75.4	-1.57841	0.698884	1.627467	0.497625
4	S005	90	79	95	98	90.5	90.4	0.742781	-0.913926	1.278724	1.030796

Looking at the table above, any score closes to 0 indicates the student performed similarly to the class average. A higher (positive) standardized score means the student performed significantly better than the class average, while a lower (negative) standardized score indicates they performed significantly worse than the rest of the class.

Final Grades Based on Weighted Average

Let's use the weighted average that we calculated earlier to assign a final grade to each student using the following grading scale:

- **A:** 85 and above
- **B:** 75 to 84.99
- **C:** 65 to 74.99
- **D:** 55 to 64.99
- **F:** Below 55

```
import numpy as np

# Define a function to assign a grade based on the weighted average
def assign_grade(weighted_avg):
    if weighted_avg >= 85:
        return 'A'
    elif weighted_avg >= 75:
        return 'B'
    elif weighted_avg >= 65:
        return 'C'
    elif weighted_avg >= 55:
        return 'D'
    else:
        return 'F'

# Apply the grading function to the weighted averages
df_result['Final_Grade'] = np.array([assign_grade(avg) for avg in
df_result['Weighted_Average']])
df_result
```

	Student_ID	Mathematics	Science	English	Information Technology	Average	Weighted Average	Final_Grade
0	S001	85	78	90	89	85.5	85.4	A
1	S002	75	88	85	65	78.25	77.6	B
2	S003	95	82	80	88	86.25	88	A
3	S004	65	85	70	92	78	75.4	B
4	S005	90	79	95	98	90.5	90.4	A

Window Functions

A **window function** performs calculations **across a set of rows** (a “window”) that are related to the current row.

The followings are the key difference between

Regular aggregation	Window functions
Groups rows together	Keeps all rows
Returns ONE row per group	Adds calculated columns
Loses individual row details	Preserves individual row details

Create a table “**Data**” with the following attribute and insert the data

Employee_ID	Employee	Department	Salary
1	Michael	Marketing	15000
2	Eyob	Marketing	18000
3	Solomon	Sales	16000
4	Esayas	Sales	13000
5	Dawit	Sales	12000

```
INSERT INTO Data (Employee_ID, Employee, Department, Salary) VALUES
(NULL, 'Michael', 'Marketing', '15000'), (NULL, 'Eyob', 'Marketing',
'18000'), (NULL, 'Solomon', 'Sales', '16000'), (NULL, 'Esayas',
'Sales', '13000'), (NULL, 'Dawit', 'Sales', '12000');
```

Aggregate Functions

Aggregate window functions calculate aggregates over a window of rows while retaining individual rows. Common aggregate functions include:

- **SUM()**: Sums values within a window.
- **AVG()**: Calculates the average value within a window.
- **COUNT()**: Counts the rows within a window.
- **MAX()**: Returns the maximum value in the window.
- **MIN()**: Returns the minimum value in the window.

Think of a regular aggregate function that request to get an average salary and their rank based on ascending salary for each department.

```
SELECT Department, AVG(Salary)
FROM Employees
GROUP BY Data;
```

Department	AVG(Salary)
Marketing	16500.0000
Sales	13666.6667

Result: One row per department without the individual employee details!

```
SELECT
    Employee,
    Department,
    Salary,
    AVG(Salary) OVER (PARTITION BY Department) AS Department_Average
FROM Data;
```

Employee	Department	Salary	Department_Average
Eyob	Marketing	18000	16500.0000
Michael	Marketing	15000	16500.0000
Dawit	Sales	12000	13666.6667
Solomon	Sales	16000	13666.6667
Esayas	Sales	13000	13666.6667

Result: Every employee data PLUS their department based on their salary.

Ranking Functions

These functions provide rankings of rows within a partition based on specific criteria. Common ranking functions include:

- **RANK():** Assigns ranks to rows, skipping ranks for duplicates.
- **DENSE_RANK():** Assigns ranks to rows without skipping rank numbers for duplicates.
- **ROW_NUMBER():** Assigns a unique number to each row in the result set.
- **PERCENT_RANK():** Shows the relative rank of a row as a percentage between 0 and 1.

RANK() Function

RANK() functions are used to assign ranks to rows within a group based on a specific order. Ranking functions are commonly used to organize and analyze data. Some common ranking functions include:

Example: Using RANK(), we will rank employees by salary, allowing gaps in ranks when salaries are equal.

After adding the query to inserting two value with same salary

```
INSERT INTO `Data` (`Employee_ID`, `Employee`, `Department`,  
`Salary`) VALUES (NULL, 'Samuel', 'Marketing', '18000'),  
(NULL, 'Mahlet', 'Sales', '16000');
```

```
SELECT Employee, Department, Salary,  
       RANK() OVER(PARTITION BY Department ORDER BY Salary  
DESC) AS Employee_Rank  
FROM Data;
```

Employee	Department	Salary	Employee_Rank
Eyob	Marketing	18000	1
Samuel	Marketing	18000	1
Michael	Marketing	15000	3
Solomon	Sales	16000	1
Mahlet	Sales	16000	1
Esayas	Sales	13000	3
Dawit	Sales	12000	4

- RANK() function assigns a ranking within each department based on salary (highest salary = rank 1 both for Eyob and Solomon).
- Salary: Eyob and Samuel both earn 18000, so both get rank 1. The next salary is 15,000, so it gets rank 3 (rank 2 is skipped).

DENSE RANK() Function

DENSE_RANK() gives the same rank to rows with equal values. It then continues with the next number without skipping, keeping the ranking sequence continuous.

Example: Using DENSE_RANK(), we will rank employees by salary without skipping ranks.

```
SELECT Employee, Department, Salary,  
       DENSE_RANK() OVER(PARTITION BY Department ORDER BY Salary  
DESC) AS Employee_Dense_Rank  
FROM Data;
```

Employee	Department	Salary	Employee_Dense_Rank
Eyob	Marketing	18000	1
Samuel	Marketing	18000	1
Michael	Marketing	15000	2
Solomon	Sales	16000	1
Mahlet	Sales	16000	1
Esayas	Sales	13000	2
Dawit	Sales	12000	3

- DENSE_RANK() works like RANK(), but it ensures the ranking sequence has no gaps.
- Finance: Eyob and Samuel both earn 18,000, so they get rank 1. The next salary (15,000) is assigned rank 2 (no gap).

ROW_NUMBER() Function

ROW_NUMBER() gives a unique number to each row in the result set. It increments by 1 for every row, even if values are the same, so no two rows have the same number. Using ROW_NUMBER(), we will assign a unique number to each row based on salary order.

```
SELECT Employee, Department, Salary,
       ROW_NUMBER() OVER(PARTITION BY Department ORDER BY Salary DESC) AS Employee_Serial
FROM Data;
```

Employee	Department	Salary	Employee_Serial
Eyob	Marketing	18000	1
Samuel	Marketing	18000	2
Michael	Marketing	15000	3
Mahlet	Sales	16000	1
Solomon	Sales	16000	2
Esayas	Sales	13000	3
Dawit	Sales	12000	4

ROW_NUMBER() works like the above but it ensures the ranking sequence has no gaps irrespective of the Salary.

PERCENT RANK() Function

PERCENT_RANK() shows where a row stands compared to others in the same group. The formula is:

$$\text{PERCENT_RANK}() = \frac{(\text{RANK} - 1)}{\text{Total Rows in Partition} - 1}$$

Example: Using PERCENT_RANK(), we will find the relative salary position of each employee within a department.

```
SELECT Employee, Department, Salary,  
       PERCENT_RANK() OVER(PARTITION BY Department ORDER BY  
Salary DESC) AS Employee_percent_rank  
FROM Data;
```

Employee	Department	Salary	Employee_percent_rank
Eyob	Marketing	18000	0.0000000000
Samuel	Marketing	18000	0.0000000000
Michael	Marketing	15000	1.0000000000
Solomon	Sales	16000	0.0000000000
Mahlet	Sales	16000	0.0000000000
Esayas	Sales	13000	0.6666666667
Dawit	Sales	12000	1.0000000000

In Salary: Eyob & Samuel are tied for highest = 0.00, Michael (lowest) = 1.00. Each value shows the employee's relative position within the department.

NoSQL basics

NoSQL databases are built to manage large amounts of unstructured and semi-structured data with flexible schemas and high scalability.

NoSQL databases, often referred to as "**Not Only SQL**," provide non-relational data management focused on high scalability and flexible schemas. Understanding their role involves contrasting them with traditional SQL foundations and applying Exploratory Data Analysis (EDA) to their often-unstructured data. The following are the characteristics of NoSQL

- Do not rely on fixed tables or predefined schemas.
- Support flexible data models for diverse data types.
- Enable horizontal scaling for high performance.

- Well-suited for modern, data-intensive applications.

Over time, four major types of NoSQL databases have emerged: **document databases**, **key-value databases**, **wide-column stores**, and **graph databases**. Nowadays, multi-model databases are also becoming quite popular.

Document-oriented databases

A document-oriented database stores data in documents similar to JSON (JavaScript Object Notation) objects. Each document contains pairs of fields and values. The values can typically be a variety of types, including things like strings, numbers, booleans, arrays, or even other objects. A document database offers a flexible data model, much suited for semi-structured and typically unstructured data sets. They also support nested structures, making it easy to represent complex relationships or hierarchical data. Examples of document databases are MongoDB and Couchbase. A typical document will look like the following:

```
{
  "_id": "12345",
  "name": "foo bar",
  "email": "foo@bar.com",
  "address": {
    "street": "123 foo street",
    "city": "some city",
    "state": "some state",
    "zip": "123456"
  },
  "hobbies": ["music", "guitar", "reading"]
}
```

Key-value databases

A key-value store is a simpler type of database where each item contains keys and values. Each key is unique and associated with a single value. They are used for caching and session management and provide high performance in reads and writes because they tend to store things in memory. Examples are Amazon DynamoDB and Redis. A simple view of data stored in a key-value database is given below:

```
Key: user:12345
```

```
Value: {
  "name": "Solomon Yirga",
  "email": "solomon.yirga@gmail.com",
  "position": "Software Engineer"
}
```

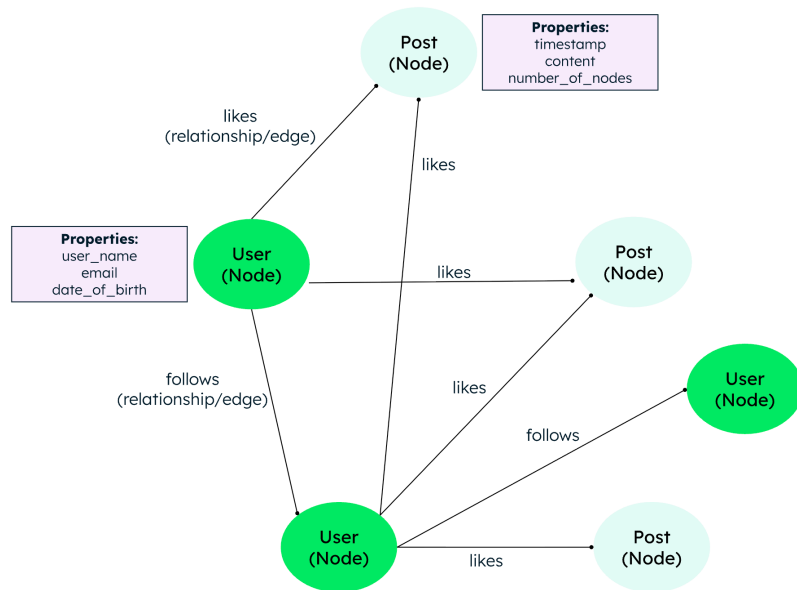
Wide-column stores

Wide-column stores store data in tables, rows, and dynamic columns. The data is stored in tables. However, unlike traditional SQL databases, wide-column stores are flexible, where different rows can have different sets of columns. These databases can employ column compression techniques to reduce the storage space and enhance performance. The wide rows and columns enable efficient retrieval of sparse and wide data. Some examples of wide-column stores are Apache Cassandra and HBase. A typical example of how data is stored in a wide-column.

Full Name	ID	Email	DoB	City
Solomon Yigzaw	12345	solomon.yigzaw@gmail.com		Some city
Kaleb Tadesse	34521	kaleb.tadesse@gmail.com	1972-05-12	

Graph databases

A graph database stores data in the form of nodes and edges. Nodes typically store information about people, places, and things (like nouns), while edges store information about the relationships between the nodes. They work well for highly connected data, where the relationships or patterns may not be very obvious initially. MongoDB also provides graph traversal capabilities using the \$graphLookup stage of the aggregation pipeline. Below is an example of how data is stored:



Multi-model databases

Multi-model databases support more than one type of NoSQL data model so that developers can choose based on their application requirements. These databases have a unified database engine that can handle multiple data models within a database instance. Examples are CosmosDB and ArangoDB.

